



Universidad Nacional Autónoma de México

Facultad de Estudios Superiores Aragón



Vinculación empresarial

Proyecto

Sistema de incidencias

Alumno:

Jaime Abraham Morales Frausto

Maestro:

Aaron Velasco Agustin

Grupo:

2007

2. Índice

- 2. Índice
- 3. Resumen Ejecutivo
- 4. Introducción
- 5. Planteamiento del Problema
- 6. Objetivos
- 7. Marco Teórico
- 8. Alcance del Sistema
- 9. Requerimientos del Sistema
- 10. Casos de Uso
- 11. Historias de Usuario
- 12. Tecnologías Seleccionadas
- 13. Arquitectura del Sistema
- 14. Estructura del Proyecto y Distribución
- 15. Uso de Contenedores y Despliegue
- 16. Modelo de Base de Datos
- 17. Estrategia de Control de Versiones
- 18. Seguridad del Sistema
- 19. Pruebas y Calidad
- 20. Riesgos del Proyecto
- 21. Conclusiones
- 22. Referencias
- A. Matriz de Trazabilidad
- B. Evidencias y Repositorio
- C. Recomendaciones de Mejora

3. Resumen Ejecutivo

3.1 Problema Identificado

Las instituciones educativas de nivel medio superior y superior enfrentan de manera recurrente la dificultad de gestionar eficazmente las incidencias físicas que se presentan en sus instalaciones: aulas, laboratorios, talleres, sanitarios y áreas comunes. La ausencia de un canal formal, centralizado y trazable para el registro, seguimiento y resolución de dichas incidencias provoca pérdida de información, retrasos en la atención y falta de rendición de cuentas por parte de las áreas responsables.

3.2 Solución Propuesta

Andevia es un sistema web de gestión y control de reportes de inspección de instalaciones físicas. Permite al personal de la institución crear, consultar y dar seguimiento a reportes de incidencias vinculados a espacios físicos concretos, asignarlos a áreas responsables y técnicos, registrar evaluaciones estandarizadas por tipo de espacio y visualizar analíticas de desempeño operativo en tiempo real.

3.3 Tecnologías Seleccionadas

Capa	Tecnología
Framework principal	Next.js 16.2.1 (App Router, React 19)
Lenguaje	TypeScript 5
Base de datos	MySQL 8 (contenedor Docker)
ORM	Prisma 6
Autenticación	NextAuth.js v5 (beta)
Formularios y validación	React Hook Form + Zod 4
Interfaz de usuario	shadcn/ui + Tailwind CSS v4 + Radix UI
Estado global	Zustand 5
Exportación	SheetJS (xlsx)
Contenerización	Docker + Docker Compose

3.4 Beneficios Esperados

- Centralización del registro de incidencias con trazabilidad completa.
- Reducción del tiempo de respuesta mediante asignación automática y seguimiento de estado.
- Visibilidad gerencial a través de analíticas e indicadores clave de desempeño (KPIs).
- Estandarización de criterios de evaluación diferenciados por tipo de espacio.
- Reducción de gestión manual y eliminación del uso de formularios en papel.

3.5 Alcance General

El sistema abarca la gestión completa del ciclo de vida de un reporte: creación, asignación, seguimiento, resolución y análisis estadístico. Contempla tres roles de usuario (Administrador, Jefe de Área / STAFF, Técnico), 27 grupos de ubicaciones, más de 400 espacios físicos catalogados y 7 áreas de responsabilidad institucional.

4. Introducción

4.1 Contexto del Problema

La infraestructura física de una institución educativa constituye uno de los factores determinantes en la calidad del proceso enseñanza-aprendizaje. Espacios limpios, funcionales y adecuadamente equipados inciden directamente en el rendimiento académico y en el bienestar de la comunidad escolar. Sin embargo, la administración eficiente de dichos espacios representa un reto operativo de considerable complejidad, especialmente en instituciones con una planta física extensa, compuesta por múltiples bloques académicos, laboratorios especializados, talleres, áreas administrativas y zonas comunes.

4.2 Situación Actual

Previo a la implementación de Andevia, el flujo de reporte de incidencias en la institución se apoyaba en mecanismos informales: comunicación verbal entre docentes y personal administrativo, mensajería instantánea no estructurada y formularios físicos sin mecanismo de seguimiento. Este enfoque resultaba en la pérdida frecuente de solicitudes, imposibilidad de conocer el estado de una incidencia en tiempo real, falta de asignación clara de responsabilidades y ausencia de métricas históricas que permitieran identificar áreas o espacios con mayor índice de problemas recurrentes.

4.3 Importancia de la Digitalización

La transformación digital de los procesos administrativos en instituciones educativas es una necesidad reconocida tanto por organismos internacionales como la UNESCO y la OCDE, como por el marco normativo nacional de la educación superior en México. La digitalización de la gestión de instalaciones permite no sólo optimizar tiempos de respuesta sino también generar evidencia cuantificable del cumplimiento de estándares de mantenimiento, requerida en procesos de acreditación y certificación institucional.

4.4 Justificación del Sistema

El desarrollo de Andevia se justifica a partir de la necesidad de contar con un sistema de información que centralice, estandarice y automatice el proceso de reporte, seguimiento y resolución de incidencias físicas. La elección de una arquitectura web (accesible desde cualquier dispositivo con navegador) garantiza la adopción por parte de todo el personal, independientemente de su perfil técnico o del equipo disponible.

4.5 Objetivo General del Proyecto

Desarrollar e implementar un sistema web de gestión de reportes de inspección de instalaciones físicas, que permita al personal de la institución registrar, dar seguimiento y resolver incidencias en aulas, laboratorios, sanitarios y áreas comunes, facilitando la toma de decisiones operativas mediante analíticas en tiempo real.

5. Planteamiento del Problema

5.1 Situación Actual

La institución cuenta con una planta física compuesta por 27 grupos de ubicaciones (bloques académicos, laboratorios, talleres, servicios sanitarios, áreas administrativas y zonas comunes), sumando más de 400 espacios físicos individuales. La gestión de incidencias en estos espacios se realiza de forma fragmentada, sin una plataforma unificada.

5.2 Problemas Detectados

- Ausencia de un canal formal único para el registro de incidencias físicas.
- Falta de trazabilidad: los reportes verbales o por mensajería no quedan registrados permanentemente.
- Duplicidad de reportes: distintos actores reportan la misma incidencia sin conocer que ya existe un reporte previo.
- Desconocimiento del estado de atención: el solicitante no tiene visibilidad sobre si su reporte fue recibido, asignado o resuelto.
- Asignación de responsabilidades difusa: no existe un mecanismo claro para derivar una incidencia al área y técnico adecuados.
- Carencia de indicadores históricos: no se cuenta con datos estadísticos sobre tiempos de respuesta, incidencias recurrentes o áreas con mayor carga de trabajo.

5.3 Impacto Operativo

La ausencia de un sistema estructurado genera costos operativos ocultos: tiempo del personal dedicado a la búsqueda de información, repetición de esfuerzos, deterioro de la infraestructura por falta de atención oportuna y quejas de docentes y alumnos que afectan el clima institucional.

5.4 Consecuencias de No Implementar la Solución

De no implementarse un sistema de gestión de incidencias, la institución continuará operando con procesos ineficientes que impiden una gestión proactiva del mantenimiento. En el mediano plazo, el deterioro acumulado de la infraestructura puede traducirse en costos de reparación significativamente mayores a los que habría implicado una atención oportuna, además de posibles riesgos a la seguridad de la comunidad escolar.

6. Objetivos

6.1 Objetivo General

Desarrollar un sistema web integral de gestión de reportes de inspección de instalaciones físicas escolares, que automatice el ciclo completo de vida de una incidencia —desde su registro hasta su resolución— proporcionando trazabilidad, asignación de responsabilidades y analíticas de desempeño para la toma de decisiones institucionales.

6.2 Objetivos Específicos

1. Implementar un módulo de creación de reportes con evaluaciones estandarizadas diferenciadas por categoría de espacio (salones, sanitarios y áreas comunes), incluyendo criterios cuantitativos y cualitativos para cada tipo.
2. Desarrollar un sistema de gestión de estados de incidencia (PENDIENTE, EN_PROCESO, ATENDIDO) con registro automático de marcas de tiempo para cada transición, permitiendo el seguimiento preciso del ciclo de atención.
3. Diseñar e implementar un módulo de asignación de responsabilidades que permita vincular cada reporte a un área institucional y a un técnico responsable, con visibilidad diferenciada según el rol del usuario autenticado.
4. Integrar un sistema de autenticación y autorización basado en roles (ADMIN, STAFF, TECNICO) con control de acceso granular a nivel de rutas, APIs y datos, garantizando que cada usuario acceda únicamente a la información pertinente a su función.
5. Desarrollar un módulo de analíticas con indicadores clave de desempeño (KPIs) sobre el estado y evolución de los reportes, incluyendo tendencias temporales, distribución por área responsable, por edificio y por tipo de espacio, con selección de rangos de fecha personalizados.
6. Implementar una bitácora de auditoría (log de acciones) que registre automáticamente cada cambio significativo sobre un reporte, incluyendo el usuario responsable, la acción realizada, la dirección IP y el dispositivo utilizado.
7. Proveer funcionalidades de exportación de reportes en formato Excel (.xlsx) para facilitar el análisis externo y la generación de informes institucionales.

7. Marco Teórico

7.1 Sistemas de Información

Un sistema de información (SI) es un conjunto organizado de elementos interrelacionados —personas, datos, procesos y tecnología— cuya finalidad es recopilar, procesar, almacenar y distribuir información para apoyar la toma de decisiones y el control en una organización (Laudon & Laudon, 2020). Andevia encuadra en la categoría de Sistema de Información Operacional (SIO), ya que soporta las actividades cotidianas de registro y seguimiento de incidencias, y en Sistema de Soporte a la Toma de Decisiones (DSS) en su módulo de analíticas, que provee indicadores agregados para la gestión institucional.

7.2 Sistemas Administrativos

Los sistemas administrativos son aplicaciones de software diseñadas para automatizar y optimizar procesos organizacionales. En el contexto de la gestión de instalaciones físicas, se enmarcan en la categoría de Facility Management Systems (FMS) o Sistemas de Gestión de Instalaciones, los cuales tienen como objetivo principal garantizar la funcionalidad, el confort y la eficiencia de los espacios físicos (IFMA, 2021). Andevia implementa las funcionalidades básicas de un FMS: catálogo de espacios, registro de incidencias, asignación de tareas de mantenimiento y generación de reportes.

7.3 Desarrollo Web

El desarrollo web moderno se sustenta en el paradigma de aplicaciones web de página única (Single Page Applications, SPA) y aplicaciones de renderizado del lado del servidor (Server-Side Rendering, SSR). Next.js, framework utilizado en Andevia, combina ambos paradigmas mediante su App Router, que permite definir páginas con renderizado en servidor (React Server Components) para la carga inicial y componentes del lado del cliente para interactividad. Esta arquitectura híbrida optimiza tanto el rendimiento percibido (Core Web Vitals) como la experiencia de desarrollo.

7.4 Bases de Datos

MySQL 8 es un Sistema de Gestión de Bases de Datos Relacionales (SGBDR) de código abierto, ampliamente adoptado en entornos de producción por su estabilidad, rendimiento y soporte para transacciones ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad). El acceso a la base de datos en Andevia se realiza exclusivamente a través de Prisma ORM, que proporciona un cliente de base de datos type-safe generado a partir del esquema declarativo definido en `schema.prisma`, eliminando el riesgo de inyecciones SQL directas y mejorando la mantenibilidad del código.

7.5 Gestión de Incidencias

La gestión de incidencias es una práctica formal derivada del marco ITIL (Information Technology Infrastructure Library), que define los procesos para identificar, registrar, clasificar, asignar, escalar y resolver perturbaciones en los servicios. Si bien ITIL se origina en el ámbito de TI, sus principios son transferibles a la gestión de instalaciones físicas. Andevia implementa los elementos esenciales: registro estructurado, categorización por tipo de espacio, asignación a grupos de soporte (áreas), actualización de estado y cierre con registro de resolución.

7.6 Control de Versiones con Git

Git es el sistema de control de versiones distribuido estándar de la industria del desarrollo de software. Permite rastrear cambios en el código fuente, colaborar en equipo mediante ramas (branches) y revertir modificaciones cuando sea necesario. El proyecto Andevia utiliza Git para el control de versiones, siguiendo buenas prácticas de desarrollo como la separación de funcionalidades en ramas independientes y el uso de mensajes de commit descriptivos.

7.7 Contenedores Docker

Docker es una plataforma de contenerización que permite empaquetar una aplicación junto con todas sus dependencias en una unidad portable y reproducible denominada contenedor. A diferencia de las máquinas virtuales, los contenedores comparten el kernel del sistema operativo anfitrión, lo que los hace significativamente más ligeros y eficientes. Docker Compose es la herramienta complementaria que permite definir y orquestar aplicaciones multi-contenedor mediante un archivo YAML declarativo. Andevia utiliza Docker Compose para orquestar dos servicios: la aplicación Next.js y la base de datos MySQL 8.

7.8 Arquitectura Cliente-Servidor

La arquitectura cliente-servidor es el modelo computacional predominante en aplicaciones web. En este modelo, el cliente (navegador web) realiza solicitudes a través del protocolo HTTP/HTTPS al servidor, que procesa dichas solicitudes y retorna respuestas. Next.js con App Router implementa una variante avanzada de este modelo: los React Server Components se ejecutan en el servidor y envían HTML renderizado al cliente, reduciendo el JavaScript enviado al navegador, mientras que los Client Components permiten interactividad local sin viajes de red innecesarios.

7.9 APIs REST

REST (Representational State Transfer) es un estilo arquitectónico para el diseño de servicios web que utiliza los verbos HTTP (GET, POST, PUT, PATCH, DELETE) para operar sobre recursos identificados por URLs. Andevia expone una API REST interna mediante las Route Handlers de Next.js (archivos route.ts dentro del directorio app/api/), con endpoints organizados por recurso: /api/reportes, /api/personal, /api/areas, /api/espacios, /api/grupos, /api/tipos-espacio, /api/analiticas y /api/usuarios.

7.10 Seguridad Informática

La seguridad informática en aplicaciones web involucra múltiples capas de protección. Andevia implementa: (1) autenticación mediante NextAuth.js v5 con proveedor de credenciales y tokens JWT firmados con AUTH_SECRET; (2) autorización basada en roles (RBAC) a nivel de middleware de Next.js, que protege todas las rutas de dashboard; (3) hashing de contraseñas con bcryptjs (10 rounds de salt); (4) validación de entradas en servidor mediante Zod antes de persistir datos; (5) protección de variables sensibles mediante variables de entorno (.env); y (6) visibilidad de datos filtrada por área del usuario autenticado.

8. Alcance del Sistema

8.1 Alcances

- Registro de reportes de incidencias físicas vinculados a espacios concretos (aula, laboratorio, taller, sanitario, área administrativa, área común).
- Gestión del ciclo de vida completo de un reporte: PENDIENTE → EN_PROCESO → ATENDIDO, con marcas de tiempo automáticas.
- Evaluaciones estandarizadas diferenciadas por categoría: salones (6 criterios), sanitarios (7 criterios) y áreas comunes (6 criterios).
- Asignación de reportes a áreas responsables (Mantenimiento General, Electricidad, Plomería, Limpieza, Infraestructura, Tecnología) y técnicos individuales.
- Panel de control (dashboard) con estadísticas en tiempo real: total de reportes, pendientes y atendidos recientes.
- Módulo de analíticas con KPIs, tendencia temporal, distribución por área, por edificio y por tipo de espacio.
- Catálogo de 27 grupos de ubicaciones y más de 400 espacios físicos precargados.
- Gestión de personal: vista diferenciada para ADMIN (gestión completa) y STAFF (vista filtrada por área propia).
- Bitácora de auditoría automática para cambios de estado, asignación de área y asignación de responsable.
- Exportación de reportes a formato Excel (.xlsx).
- Soporte para borradores (isDraft) de reportes sin finalizar.
- Interfaz adaptable (responsiva) con soporte para modo oscuro/claro.
- Despliegue contenerizado mediante Docker Compose para entornos locales y de producción.

8.2 Limitaciones

- El sistema no incluye notificaciones automáticas por correo electrónico o mensajería push al crear o actualizar reportes.
- No contempla la carga de fotografías o adjuntos dentro de los reportes.
- No existe un módulo de calendario o agenda para la programación de inspecciones preventivas.
- La asignación de reportes a técnicos es manual; no existe lógica automática de balanceo de carga.

8.3 Exclusiones

- Integración con sistemas ERP o de nómina institucional.
- Gestión de inventario de materiales de mantenimiento.
- Módulo de facturación o control presupuestario de mantenimiento.
- Aplicación móvil nativa (iOS / Android); el acceso es exclusivamente vía navegador web.

9. Requerimientos del Sistema

3. Requerimientos Funcionales (mínimo 12)

ID	Requerimiento Funcional
RF-01	El sistema deberá contar con un ¿identificador único (ID) para cada empleado.
RF-02	El sistema deberá garantizar disponibilidad continua (24/7).
RF-03	El sistema deberá ser intuitivo y fácil de usar para el usuario.
RF-04	El sistema deberá operar con un bajo consumo de recursos.
RF-05	El sistema deberá tener la documentación adecuada.
RF-06	El sistema deberá facilitar su actualización de forma rápida y eficiente.
RF-07	El sistema deberá poder instalarse en diferentes servidores de manera sencilla y sin complicaciones.
RF-08	El sistema deberá permitir la asignación de un rango a cada empleado según su puesto, definiendo el acceso a distintas funcionalidades del sistema.
RF-09	El sistema deberá enviar notificaciones cuando un reporte sea asignado o actualizado.
RF-10	El sistema deberá permitir filtrar y buscar reportes por estado, fecha o ubicación
RF-11	El sistema deberá generar métricas y estadísticas sobre los reportes registrados.
RF-12	El sistema deberá permitir cerrar reportes una vez que el problema haya sido solucionado.

4. Requerimientos No Funcionales (mínimo 8)

ID	Requerimiento No Funcional
RNF-01	El sistema deberá solicitar llaves de acceso o ID para cada empleado al momento de iniciar labores para localizar el abuso de levantamiento de reportes
RNF-02	Estar disponible 24/7 o al menos lo requerido dentro de jornada laboral de los empleados
RNF-03	El sistema deberá contar con una interfaz sencilla e intuitiva.
RNF-04	El sistema deberá tener una baja demanda de recurso en los dispositivos en los que se use.
RNF-05	El sistema deberá contar con documentación adecuada sobre su desarrollo, estructura y funcionalidades.
RNF-06	El sistema deberá ser fácil de actualizar y de dar mantenimiento.
RNF-07	El sistema deberá ser capaz de ser instalado en dispositivos tipo smartphone y PC
RNF-08	El sistema deberá contar con funciones distintas según el papel o rol de los empleados. 1. Quienes reportan tickets 2. Quien los recibe y administra

5. Reglas de Negocio (mínimo 6)

ID	Regla de Negocio
RB-01	<u>Todo reporte debe ser creado exclusivamente por un usuario autenticado (estudiante, docente o personal técnico - administrativo) con cuenta institucional UNAM.</u>

RB-02	<u>Cada reporte debe asignarse automáticamente un ID único e irrepetible (formato: REP-YYYYMMDD-XXXX) al momento de su creación.</u>
RB-03	<u>Los reportes de urgencia “Crítica” (seguridad eléctrica, inundaciones, riesgos estructurales) deben notificarse de inmediato al administrador y al jefe de mantenimiento vía correo y dashboard (máximo 5 minutos)</u>
RB-04	<u>Un reporte no puede cambiar al estado “En curso” sin que se haya asignado un técnico responsable.</u>
RB-05	<u>Un reporte solo puede pasar a “Resuelto” si el técnico registra evidencia (notas detalladas + fecha/hora de cierre). El administrador debe validar el cierre.</u>
RB-06	<u>El tiempo de atención (MTTR) se calcula automáticamente desde el momento en que el estado cambia a “En curso” hasta “Resuelto”.</u>
RB-07	<u>Los reportes pendientes que superen 7 días naturales deben aparecer destacados en rojo en el dashboard y generar alerta automática al administrador y al director.</u>
RB-08	<u>Solo el rol “Administrador” o “Jefe de Mantenimiento” puede reasignar un reporte a otro técnico o cancelarlo.</u>
RB-09	<u>Los reportes clasificados en laboratorios L1-L4 (Ingeniería) o áreas de seguridad (baños de alta ocupación) tendrán prioridad automática sobre reportes de aulas comunes.</u>
RB-10	<u>Todo cambio de estado, asignación o nota debe quedar registrado en el log de auditoría con usuario, fecha, hora y motivo.</u>

RB-11	<u>Los reportes cerrados no pueden volver a “Pendiente” o “En curso” sin justificación por escrito del administrador (máximo una vez)</u>
RB-12	<u>El sistema debe calcular mensualmente las métricas por edificio y enviar automáticamente un resumen en PDF al director de la FES Aragón.</u>
RB-13	<u>Los reportes relacionados con accesibilidad (rampas, elevadores, baños para personas con discapacidad) deben marcarse con etiqueta especial y reportarse trimestralmente al área de Inclusión.</u>
RB-14	<u>Ningún usuario puede eliminar un reporte; sólo el Administrador puede archivar reportes cerrados después de 90 días.</u>
RB-15	<u>El sistema debe bloquear la creación de reportes duplicados en la misma ubicación con la misma descripción dentro de las siguientes 24 horas (detección automática por similitud).</u>

10. Casos de Uso

CU-01 Nombre del Caso de Uso

Actor: personal administrativo

Descripción: Permite a un usuario autenticado registrar un nuevo reporte de incidencia dentro de las instalaciones.

Precondición: El usuario debe estar autenticado en el sistema.

Debe tener una cuenta institucional válida.

1. Flujo Principal:
El usuario ingresa al sistema.
2. Selecciona la opción Crear Reporte.
3. Introduce información del reporte:
 - a. Ubicación
 - b. Descripción del problema
 - c. Nivel de urgencia
 - d. Fecha.

Flujos Alternos:

A1:El sistema valida la información.

A2:El sistema genera automáticamente un ID único de reporte.

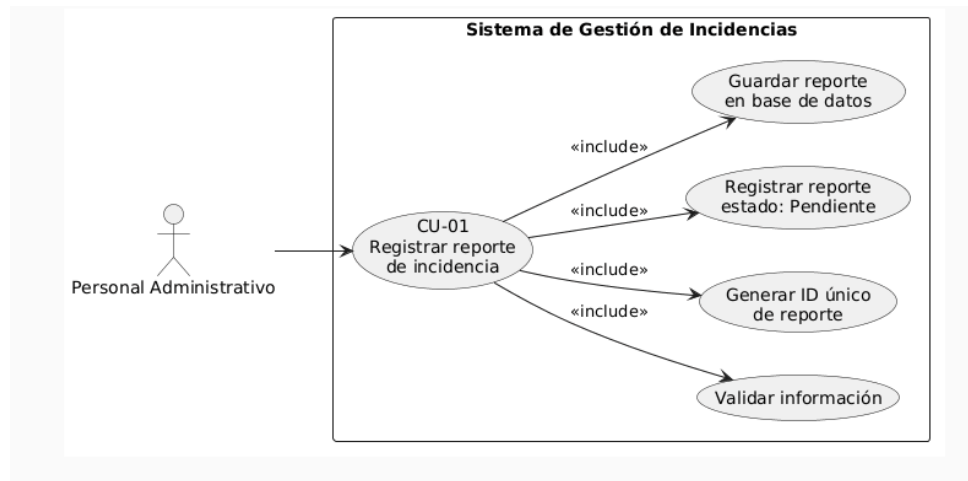
A3:El sistema registra el reporte con estado Pendiente.

A4:El sistema guarda el reporte en la base de datos.

Postcondición:

El reporte queda registrado en el sistema.

El reporte queda disponible para asignación.



CU-02 Asignar responsable

Actor: Administrador

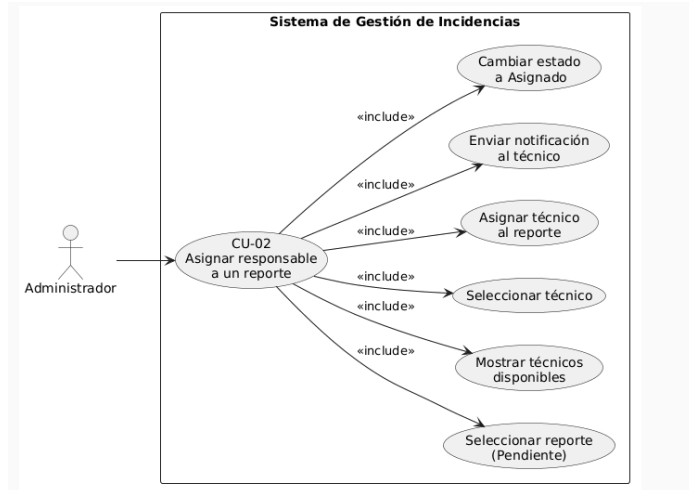
Descripción: Permite asignar un técnico responsable para atender un reporte registrado.

Precondición: Debe existir un reporte registrado.

El actor debe tener rol de administrador o jefe de mantenimiento.

1. Flujo Principal:
El administrador accede al panel de reportes.
2. Selecciona un reporte en estado Pendiente.
3. El sistema muestra los técnicos disponibles.
4. El administrador selecciona un técnico.
5. El sistema asigna el técnico al reporte.
6. El sistema envía una notificación al técnico.
7. El estado del reporte cambia a Asignado.

Postcondición: El reporte tiene un responsable asignado.



CU-03 Actualizar estado de reporte

Actor: Administrador

Descripción: Permite al Admin actualizar el estado del reporte durante el proceso de solución, recibe el aviso del personal de mantenimiento.

Precondición: El reporte debe tener un técnico asignado.

Recibió notificación de resolución.

1. Flujo Principal:
El Admin inicia sesión.
2. Accede a los reportes asignados.
3. Selecciona un reporte.
4. Cambia el estado a En curso.
5. El sistema registra la fecha y hora del cambio.

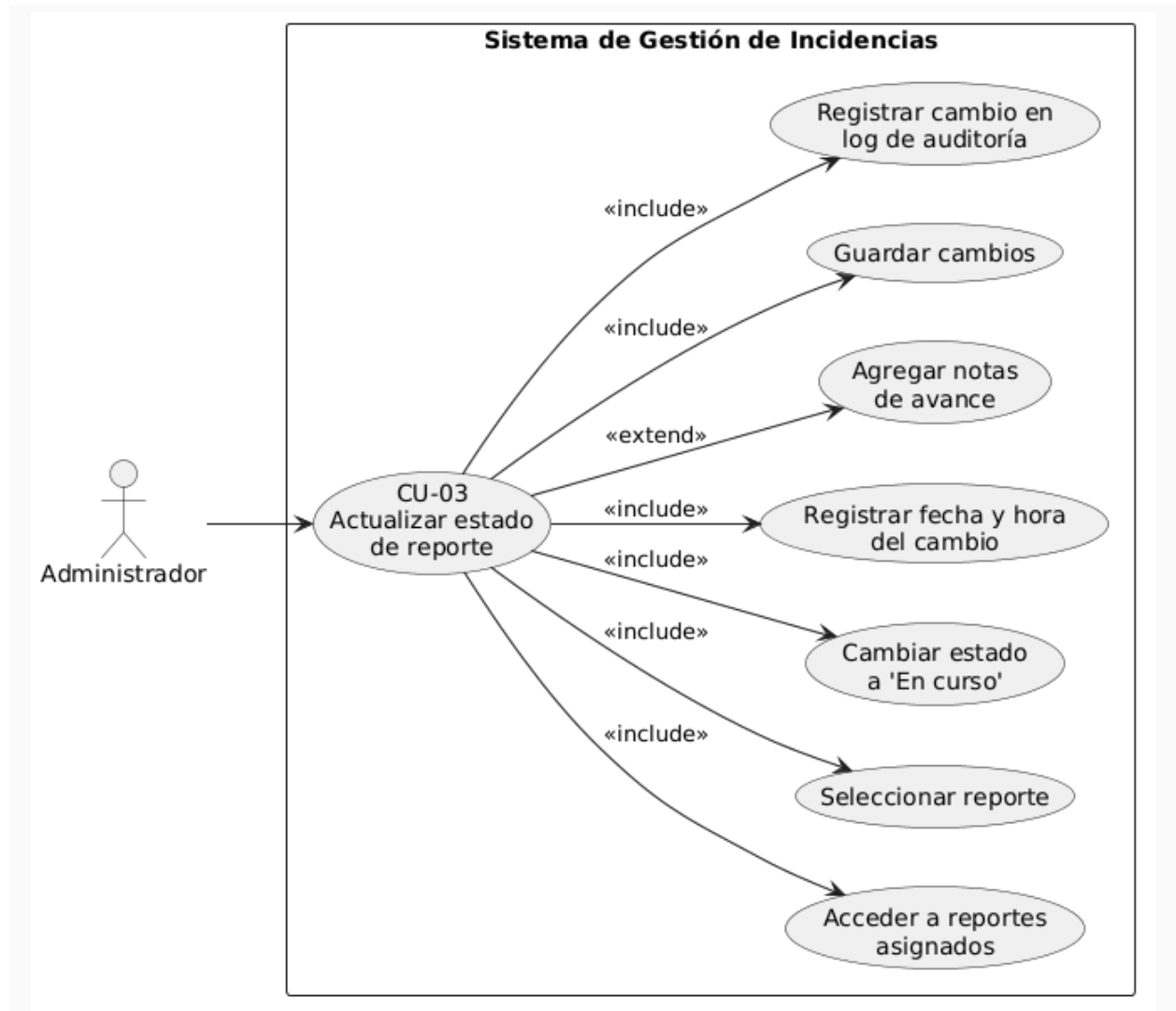
Flujos Alternos:

A1: El técnico agrega notas sobre el avance.

A2: El sistema guarda los cambios.

Postcondición: El estado del reporte queda actualizado.

El sistema registra el cambio en el log de auditoría.



CU-04 Cerrar reporte

Actor principal: Técnico / Administrador

Descripción: Permite cerrar un reporte una vez solucionada la incidencia.

Precondiciones:

El reporte debe estar en estado "En curso".

Debe existir un técnico responsable.

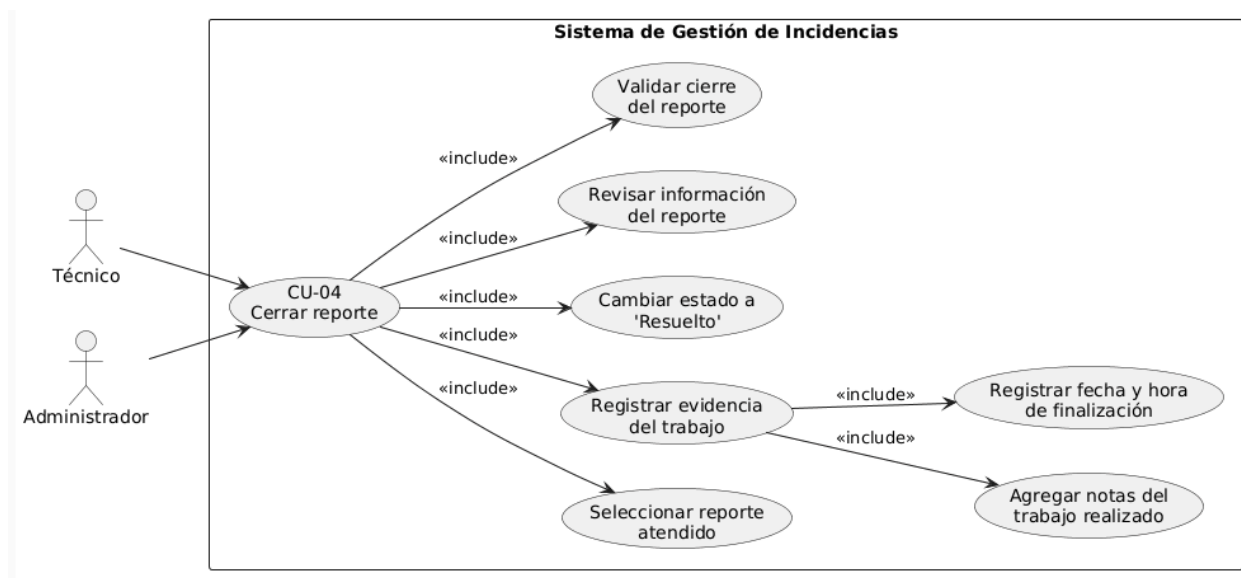
Flujo principal:

1. El técnico selecciona el reporte atendido.

2. Registra evidencia:
 - Notas del trabajo realizado
 - Fecha y hora de finalización
3. El sistema cambia el estado a "Resuelto".
4. El administrador revisa la información.
5. El administrador valida el cierre.

Postcondiciones:

El reporte queda cerrado oficialmente.



CU-05 Consultar métricas

Actor principal: Administrador / Director

Descripción: Permite visualizar estadísticas y métricas sobre los reportes registrados.

Precondiciones:

Deben existir reportes registrados en el sistema.

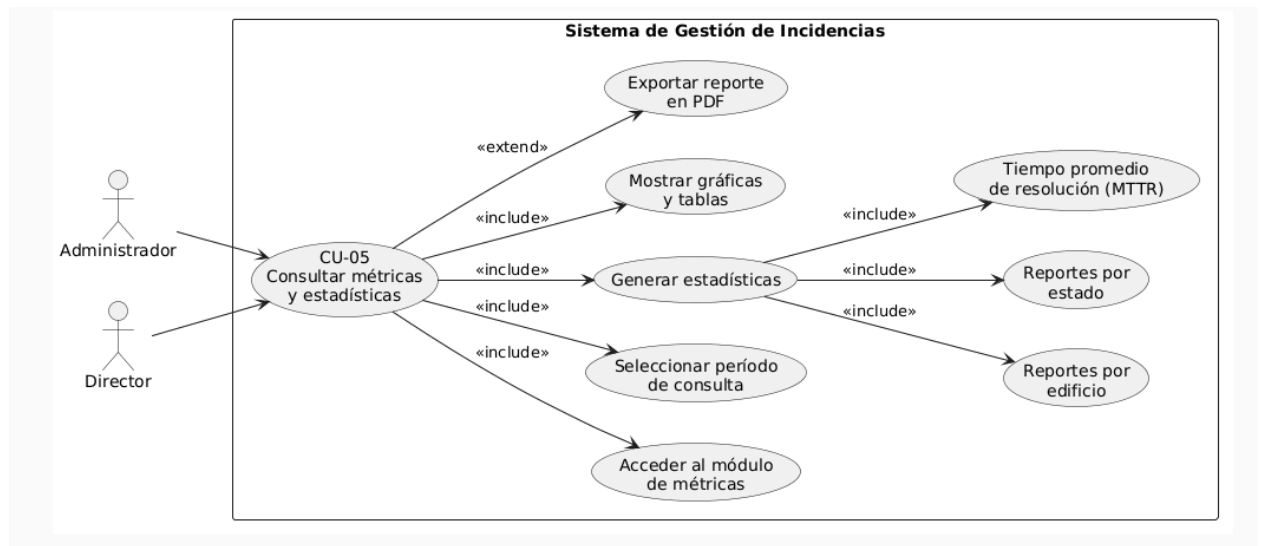
Flujo principal:

1. El administrador accede al módulo de Métricas.
2. Selecciona el período de consulta.
3. El sistema genera estadísticas como:
 - Número de reportes por edificio
 - Reportes por estado
 - Tiempo promedio de resolución (MTTR)
4. El sistema muestra gráficas y tablas.
5. El administrador puede exportar el reporte en PDF.

Postcondiciones:

- Se obtiene un reporte estadístico del sistema.

S



11. Historias de Usuario

ID	Historia de Usuario
HU-01	Como usuario institucional (estudiante, docente o administrativo) quiero registrar un reporte de incidencia en las instalaciones para informar problemas que requieran mantenimiento..
HU-02	Como usuario quiero consultar el estado de mis reportes para dar seguimiento a la solución del problema reportado.
HU-03	Como administrador del sistema quiero visualizar todos los reportes registrados para tener control y seguimiento de las incidencias.
HU-04	Como administrador o jefe de mantenimiento quiero asignar técnicos responsables a los reportes para asegurar que cada incidencia sea atendida por el personal correspondiente.
HU-05	Como técnico de mantenimiento quiero ver los reportes que me han sido asignados para organizar y atender mis tareas de mantenimiento.
HU-06	Como técnico quiero actualizar el estado de un reporte (pendiente, en curso, resuelto) para informar el progreso de la reparación o mantenimiento.
HU-07	Como administrador quiero recibir alertas de reportes críticos o urgentes para actuar rápidamente ante situaciones que representen riesgo.
HU-08	Como administrador quiero consultar métricas y estadísticas de los reportes para evaluar el desempeño del área de mantenimiento.
HU-09	Como administrador quiero filtrar y buscar reportes por estado, fecha o ubicación para encontrar información de manera rápida y organizada.
HU-10	Como director o administrador quiero generar reportes mensuales en PDF sobre incidencias para analizar la situación de mantenimiento de las instalaciones.

12. Tecnologías Seleccionadas

12.1 Framework Principal: Next.js 16.2.1

Next.js es el framework de React para producción desarrollado por Vercel. En su versión 16, integra el App Router basado en React Server Components, que permite un modelo de renderizado híbrido: las páginas se renderizan en el servidor para la carga inicial (mejorando SEO y tiempo de primera carga) y los componentes interactivos se hidratan en el cliente. En Andevia, el App Router organiza la aplicación en rutas de grupo: (auth) para páginas de autenticación y (dashboard) para el área protegida, con middleware global para la verificación de sesiones.

12.2 TypeScript 5

TypeScript es el superconjunto de JavaScript con tipado estático desarrollado por Microsoft. Su uso en Andevia garantiza la detección de errores en tiempo de compilación, mejora la productividad mediante la autocompletación del IDE y hace el código más autodocumentado. El esquema de validación Zod genera tipos TypeScript directamente desde las definiciones de esquema, creando una fuente única de verdad para la estructura de los datos.

12.3 MySQL 8

MySQL 8 es la versión más reciente del SGBDR relacional de Oracle. Incluye mejoras significativas en rendimiento, soporte para ventanas de funciones (window functions), CTEs (Common Table Expressions) y JSON nativo. En Andevia, la base de datos se ejecuta en un contenedor Docker con persistencia en volumen. Se utiliza MySQL específicamente porque el campo evaluación del modelo Reporte almacena datos estructurados variables como JSON nativo.

12.4 Prisma ORM 6

Prisma es un ORM (Object-Relational Mapping) de nueva generación para Node.js y TypeScript. Su modelo de trabajo se basa en un esquema declarativo (schema.prisma) desde el cual genera automáticamente un cliente de base de datos con tipado fuerte. El cliente generado incluye autocompletación completa en el IDE para todas las entidades y sus relaciones. Adicionalmente, Prisma expone prisma.\$queryRaw para consultas SQL nativas cuando las necesidades de agregación superan las capacidades del Query Builder, como ocurre en el módulo de analíticas.

12.5 NextAuth.js v5 Beta

NextAuth.js (Auth.js v5) es la biblioteca de autenticación más adoptada para el ecosistema Next.js. En Andevia se configura con el proveedor de credenciales (usuario/contraseña), estrategia de sesión JWT y el adaptador de Prisma para persistir cuentas y sesiones en MySQL. La configuración se divide en dos archivos: auth.config.ts (con los callbacks de sesión y autorización, exportable a Edge Runtime) y src/lib/auth.ts (con el proveedor de credenciales y la lógica de bcrypt, que requiere el runtime de Node.js).

12.6 React Hook Form + Zod 4

React Hook Form es una biblioteca de gestión de formularios para React que minimiza los re-renders mediante el uso de refs no controladas. Zod 4 es una biblioteca de validación de

esquemas en TypeScript. La combinación de ambas, integrada mediante `@hookform/resolvers/zod`, provee validación síncrona en el cliente con los mismos esquemas que se utilizan en el servidor, garantizando consistencia en las reglas de negocio de los formularios.

12.7 shadcn/ui + Tailwind CSS v4 + Radix UI

shadcn/ui es una colección de componentes de interfaz de usuario de código abierto, contruidos sobre Radix UI (primitivas accesibles sin estilo) y estilizados con Tailwind CSS. A diferencia de una biblioteca de componentes tradicional, shadcn/ui se instala copiando el código fuente de cada componente al proyecto, permitiendo personalización total. Tailwind CSS v4 introduce un nuevo motor de compilación basado en CSS nativo (cascade layers) sin necesidad de PostCSS complejo. Andevia incluye más de 25 componentes de shadcn/ui: Button, Card, Table, Dialog, Select, Input, Badge, Avatar, Dropdown, Skeleton, entre otros.

12.8 Zustand 5

Zustand es una biblioteca minimalista de gestión de estado global para React, basada en hooks. En Andevia se utiliza en dos stores: `useReporteFormStore` (para persistir el estado del formulario de nuevo reporte durante la navegación multistep) y `useUIStore` (para estado de la interfaz como filtros activos en el listado de reportes).

12.9 Docker y Docker Compose

Docker Compose orquesta dos servicios: db (MySQL 8, imagen oficial) y app (Next.js, imagen construida desde el Dockerfile del proyecto). El Dockerfile utiliza la imagen base `node:20-alpine` para minimizar el tamaño de la imagen final. El `entrypoint.sh` implementa la lógica de inicialización: espera a que MySQL esté disponible (usando `netcat`), ejecuta `prisma db push` para sincronizar el esquema y `npm run db:seed` para la inicialización de datos.

Tecnología / Librería	Versión	Propósito en el Proyecto
Next.js	16.2.1	Framework fullstack principal (routing, SSR, API)
React	19.2.4	Biblioteca de UI
TypeScript	5.x	Tipado estático
MySQL	8	Base de datos relacional
Prisma	6.19.2	ORM y gestión de esquema
NextAuth.js	5.0.0-beta.30	Autenticación y gestión de sesiones
bcryptjs	3.0.3	Hashing de contraseñas
Zod	4.3.6	Validación de esquemas
React Hook Form	7.72.0	Gestión de formularios
shadcn/ui	Latest	Componentes de UI accesibles
Tailwind CSS	4.x	Framework CSS utilitario
Radix UI	1.4.3	Primitivas de UI accesibles
Zustand	5.0.12	Estado global del cliente
Recharts	3.8.1	Gráficas y visualizaciones de datos
date-fns	4.1.0	Manipulación y formateo de fechas

Tecnología / Librería	Versión	Propósito en el Proyecto
SheetJS (xlsx)	0.18.5	Exportación a archivos Excel
Cloudinary	2.9.0	Gestión de imágenes (integración preparada)
Lucide React	1.7.0	Iconografía vectorial
Docker	Latest	Contenerización de servicios
Docker Compose	Latest	Orquestación de contenedores

13. Arquitectura del Sistema

13.1 Descripción General

Andevia implementa una arquitectura monolítica modular basada en el patrón fullstack de Next.js con App Router. En este modelo, tanto el frontend (componentes React) como el backend (Route Handlers de API y Server Components) coexisten dentro de la misma base de código y se despliegan como una única aplicación Node.js. La separación de responsabilidades se logra mediante la distinción entre Server Components (renderizado en servidor, acceso directo a Prisma), Client Components (interactividad del navegador) y Route Handlers (endpoints de API REST).

13.2 Capas del Sistema

Capa	Tecnología	Responsabilidad
Presentación (Server)	React Server Components	Renderizado inicial de páginas, obtención de datos en servidor
Presentación (Client)	React Client Components + Zustand	Interactividad, formularios, filtros, modales
API REST	Next.js Route Handlers (route.ts)	Endpoints HTTP para operaciones CRUD
Lógica de Negocio	TypeScript (lib/ y validations/)	Validación con Zod, analíticas SQL, utilidades
Autenticación	NextAuth.js v5 + Middleware	JWT, sesiones, protección de rutas
Acceso a Datos	Prisma Client	ORM, consultas type-safe, transacciones
Base de Datos	MySQL 8 (Docker)	Persistencia relacional de todos los datos

13.3 Diagrama de Flujo de Comunicación

```
sequenceDiagram
    Browser->>+Next.js Middleware: HTTP Request
    Next.js Middleware->>NextAuth: Verificar JWT
    NextAuth-->>Next.js Middleware: Sesión válida / inválida
    Next.js Middleware-->>Browser: Redirect /login (si no auth)
    Next.js Middleware->>+Server Component: Render página
    Server Component->>+Prisma: Query a DB
    Prisma->>+MySQL: SQL Query
    MySQL-->>-Prisma: ResultSet
    Prisma-->>-Server Component: Datos tipados
    Server Component-->>-Browser: HTML renderizado
    Browser->>+API Route Handler: fetch() / SWR
    API Route Handler->>NextAuth: auth() session check
    API Route Handler->>Prisma: CRUD operations
    Prisma->>MySQL: SQL
    API Route Handler-->>-Browser: JSON response
```

13.4 Endpoints de la API REST

Endpoint	Métodos	Descripción	Roles Permitidos
/api/reportes	GET, POST	Listado paginado y creación de reportes	Todos
/api/reportes/[id]	GET, PATCH, DELETE	Detalle, actualización y eliminación de reporte	Todos / ADMIN
/api/personal	GET, POST	Listado y creación de personal	Todos
/api/personal/[id]	GET, PATCH, DELETE	Detalle y gestión de personal	ADMIN, STAFF
/api/areas	GET, POST	Listado y creación de áreas	Todos
/api/areas/[id]	GET, PATCH, DELETE	Gestión de área	ADMIN
/api/espacios	GET, POST	Listado y creación de espacios físicos	Todos
/api/espacios/[id]	GET, PATCH, DELETE	Gestión de espacio	ADMIN
/api/grupos	GET, POST	Listado y creación de grupos	Todos
/api/grupos/[id]	GET, PATCH, DELETE	Gestión de grupo	ADMIN
/api/tipos-espacio	GET, POST	Listado y creación de tipos de espacio	Todos
/api/tipos-espacio/[id]	GET, PATCH, DELETE	Gestión de tipo de espacio	ADMIN
/api/usuarios	GET, POST	Listado y creación de usuarios del sistema	ADMIN
/api/usuarios/[id]	GET, PATCH, DELETE	Gestión de cuenta de usuario	ADMIN
/api/analiticas	GET	Datos de KPIs y gráficas con filtro de fechas	ADMIN, STAFF
/api/stats	GET	Estadísticas rápidas para el dashboard	Todos
/api/auth/[...nextauth]	GET, POST	Handlers de NextAuth (signin, signout, session)	—

14. Estructura del Proyecto y Distribución

14.1 Árbol de Directorios

```
Andevia/
├── prisma/
│   ├── schema.prisma      # Definición del modelo de datos
│   └── seed.ts            # Script de inicialización de datos
├── src/
│   ├── app/
│   │   ├── (auth)/        # Rutas de autenticación
│   │   │   └── login/page.tsx
│   │   ├── (dashboard)/   # Rutas protegidas
│   │   │   ├── dashboard/ # Panel principal
│   │   │   ├── reportes/  # Listado, detalle y nuevo reporte
│   │   │   ├── analiticas/ # Módulo de analíticas
│   │   │   ├── edificios/ # Gestión de espacios físicos
│   │   │   └── personal/  # Gestión de personal
│   │   └── api/           # Route Handlers (endpoints REST)
│   ├── components/
│   │   ├── analiticas/    # Componentes del módulo analíticas
│   │   ├── dashboard/     # Tabla, filtros, exportar, stats
│   │   ├── detalle/       # Componentes del detalle de reporte
│   │   ├── edificios/     # Gestión de ubicaciones
│   │   ├── layout/        # Header, Sidebar, Breadcrumb
│   │   ├── personal/      # Vistas de personal admin/staff
│   │   ├── reportes/      # Formulario y secciones de reporte
│   │   └── ui/            # Componentes shadcn/ui
│   ├── lib/
│   │   ├── auth.ts        # Configuración de NextAuth
│   │   ├── prisma.ts      # Singleton de PrismaClient
│   │   ├── analiticas.ts  # Lógica de analíticas SQL
│   │   ├── normalizar.ts  # Normalización de nombres
│   │   ├── utils/         # formatDate, statusUtils, exportToExcel
│   │   └── validations/   # Schemas Zod
│   ├── store/            # Zustand stores
│   └── types/            # Extensiones de tipos TypeScript
├── auth.config.ts        # Config de NextAuth para Edge Runtime
├── middleware.ts          # Protección global de rutas
├── docker-compose.yml     # Orquestación de contenedores
├── Dockerfile            # Imagen de la aplicación
└── entrypoint.sh         # Script de inicialización del contenedor
```

14.2 Responsabilidades por Módulo

Módulo	Ruta	Responsabilidad
Autenticación	src/app/(auth)/ + auth.config.ts + src/lib/auth.ts	Login, logout, gestión de tokens JWT, callbacks de sesión y roles
Dashboard	src/app/(dashboard)/dashboard/	Panel principal con estadísticas rápidas y listado paginado de reportes
Reportes	src/app/(dashboard)/reportes/ + src/components/reportes/	Formulario de creación, detalle de reporte, gestión de estado, bitácora
Analíticas	src/app/(dashboard)/analiticas/ + src/lib/analiticas.ts	KPIs, gráficas de tendencia y distribución con consultas SQL de agregación
Edificios	src/app/(dashboard)/edificios/ + src/components/edificios/	CRUD de tipos de espacio, grupos y espacios físicos
Personal	src/app/(dashboard)/personal/ + src/components/personal/	Gestión diferenciada de personal según rol del usuario
API	src/app/api/	Endpoints REST para todas las operaciones del sistema
UI Library	src/components/ui/	Componentes reutilizables de shadcn/ui
Estado Global	src/store/	Stores Zustand para formulario de reporte y UI
Utilidades	src/lib/utils/ + src/lib/validations/	Formato de fechas, exportación Excel, schemas de validación, labels de estado

15. Uso de Contenedores y Despliegue

15.1 Arquitectura de Contenedores

Docker Compose define dos servicios independientes que se comunican a través de la red interna creada automáticamente por Docker:

Servicio	Imagen	Puerto (host:contenedor)	Rol
db	mysql:8 (oficial)	3307:3306	Base de datos MySQL 8. Persiste datos en volumen mysqldata.
app	Construida desde Dockerfile local	3000:3000	Aplicación Next.js. Depende de db (condition: service_healthy).

15.2 Dockerfile — Análisis

```
FROM node:20-alpine
# Base ligera: Alpine Linux con Node.js 20 LTS (~180MB vs ~900MB en Debian)

RUN apk add --no-cache openssl netcat-openbsd
# openssl: requerido por Prisma para conexiones seguras
# netcat-openbsd: para el health-check en entrypoint.sh

COPY package*.json ./
RUN npm ci
# npm ci garantiza instalación reproducible desde package-lock.json
# Esta capa se cachea si package.json no cambia

ARG DATABASE_URL=mysql://build:build@localhost:3306/build
RUN npx prisma generate && npm run build
# prisma generate genera el cliente tipado
# npm run build compila Next.js para producción
```

15.3 Flujo de Inicialización del Contenedor (entrypoint.sh)

8. El servicio app espera a que MySQL esté disponible usando nc (netcat) en un loop.
9. Ejecuta npx prisma db push --accept-data-loss para sincronizar el esquema.
10. Ejecuta npm run db:seed para insertar datos de prueba (idempotente: verifica si ya existen usuarios).
11. Inicia el servidor de producción Next.js con npm run start.

15.4 Variables de Entorno

Variable	Valor por defecto en Compose	Descripción
DATABASE_URL	mysql://andevia:andevia@db:3306/andevia	Cadena de conexión MySQL. El host 'db' es el nombre del servicio Docker.
AUTH_SECRET	andevia-dev-secret-change-in-production	Secreto para firmar tokens JWT. DEBE cambiarse en producción.
AUTH_URL / NEXTAUTH_URL	http://localhost:3000	URL base de la aplicación para callbacks de OAuth.
NODE_ENV	production	Modo de ejecución de Node.js.

15.5 Comando de Despliegue

```
# Primera vez (construye imagen + levanta servicios):
docker compose up --build

# Reinicios posteriores:
docker compose up

# Resetear base de datos (elimina el volumen):
docker compose down -v && docker compose up
```

16. Modelo de Base de Datos

16.1 Diagrama Entidad-Relación

```
erDiagram
    User ||--o{ Reporte : crea
    User }o--|| Area : pertenece
    User ||--o{ Account : tiene
    User ||--o{ Session : tiene
    Area ||--o{ Personal : agrupa
    Area ||--o{ LogAccion : registra
    Personal ||--o{ Reporte : atiende
    Reporte ||--o{ LogAccion : genera
    Reporte }|--|| Espacio : ubicado_en
    Espacio }|--|| Grupo : agrupa
    Espacio }|--|| TipoEspacio : categoriza
```

16.2 Diccionario de Datos

Tabla: User

Campo	Tipo	Nulabl e	Descripción
id	String (CUID)	No	Identificador único generado automáticamente
name	String	Sí	Nombre completo del usuario
email	String (unique)	No	Correo electrónico (credencial de acceso)
emailVerified	DateTime	Sí	Fecha de verificación de email (NextAuth)
image	String	Sí	URL de imagen de perfil
password	String	Sí	Hash bcryptjs de la contraseña
role	Enum Role	No	Rol del usuario: ADMIN, STAFF o TECNICO
areald	Int	Sí	FK al área institucional a la que pertenece
createdAt	DateTime	No	Fecha de creación del registro
updatedAt	DateTime	No	Fecha de última actualización

Tabla: Reporte

Campo	Tipo	Nulabl e	Descripción
id	Int (autoincrement)	No	Identificador numérico del reporte
fechaCreacion	DateTime	No	Timestamp de creación (default: now())
fechaInspeccion	DateTime	No	Fecha en que se realizó la inspección física
nombreSolicitante	String (50)	No	Nombre de quien reporta la incidencia

Campo	Tipo	Nulabl e	Descripción
tipoUbicacion	String (50)	No	Categoría del espacio (SALONES, SANITARIOS, AREAS_COMUNES)
idEspacio	Int	No	FK al espacio físico inspeccionado
descripcion	String (500)	Sí	Descripción libre de la incidencia
estado	Enum EstadoReporte	No	Estado: PENDIENTE, EN_PROCESO o ATENDIDO
areaResponsable	String (50)	Sí	Nombre del área asignada para resolver la incidencia
observaciones	String (500)	Sí	Notas del técnico o jefe de área
fechaAtencion	DateTime	Sí	Timestamp automático al pasar a EN_PROCESO
fechaResolucion	DateTime	Sí	Timestamp automático al pasar a ATENDIDO
evaluacion	Json	Sí	Objeto JSON con criterios de evaluación según categoría
isDraft	Boolean	No	Indica si el reporte es un borrador (default: false)
creadoPorId	String	Sí	FK al usuario que creó el reporte
personalResponsableId	Int	Sí	FK al técnico asignado

Tabla: LogAccion

Campo	Tipo	Nulabl e	Descripción
id	Int (autoincrement)	No	Identificador de la entrada de bitácora
createdAt	DateTime	No	Timestamp automático de la acción
usuarioLogin	String (200)	No	Email del usuario autenticado que realizó la acción
usuarioReal	String (100)	Sí	Nombre real del operador (cuando difiere del login)
accion	String (200)	No	Descripción de la acción (ej. 'Estado: PENDIENTE → EN_PROCESO')
detalle	String (500)	Sí	Información adicional o contexto de la acción
ip	String (45)	Sí	Dirección IP del cliente (IPv4 o IPv6)
dispositivo	String (200)	Sí	User-Agent del navegador del cliente
areald	Int	Sí	FK al área del usuario que realizó la acción
reporteld	Int	Sí	FK al reporte sobre el que se realizó la acción

16.3 Enumeraciones

Enumeración	Valores	Uso
Role	ADMIN, STAFF, TECNICO	Rol del usuario en el sistema

Enumeración	Valores	Uso
EstadoReporte	PENDIENTE, EN_PROCESO, ATENDIDO	Estado actual del ciclo de vida del reporte
CategoriaEvaluacion	SALONES, SANITARIOS, AREAS_COMUNES	Determina los criterios de evaluación del formulario

17. Estrategia de Control de Versiones

17.1 Herramienta Utilizada

El proyecto utiliza Git como sistema de control de versiones distribuido. La presencia del archivo .gitignore (que excluye node_modules/, .next/, .env y otros archivos generados) confirma que el proyecto está bajo control de versiones desde sus etapas tempranas de desarrollo.

17.2 Archivos Ignorados por Git

Patrón en .gitignore	Justificación
.env	Contiene secretos (AUTH_SECRET, DATABASE_URL). Nunca debe versionarse.
node_modules/	Dependencias descargables desde npm. Su inclusión inflaría el repositorio.
.next/	Artefactos de compilación generados por Next.js. Reproducibles con npm run build.
src/generated/	Cliente Prisma generado. Se regenera con npx prisma generate.

17.3 Estrategia de Commits Recomendada

El proyecto, por su naturaleza de desarrollo académico individual o en equipo pequeño, se beneficia de una estrategia de commits semánticos siguiendo la convención Conventional Commits:

- feat: nueva funcionalidad (ej. feat: agregar exportación a Excel)
- fix: corrección de errores (ej. fix: validar AUTH_SECRET en entypoint)
- chore: tareas de mantenimiento (ej. chore: actualizar dependencias)
- docs: documentación (ej. docs: agregar README con credenciales)
- refactor: refactorización sin cambio de comportamiento

18. Seguridad del Sistema

18.1 Autenticación

El sistema implementa autenticación mediante NextAuth.js v5 con el proveedor de credenciales (email/password). El flujo de autenticación es el siguiente: (1) el usuario envía sus credenciales al endpoint `/api/auth/[...nextauth]`; (2) NextAuth invoca la función `authorize()` del proveedor de credenciales; (3) se valida el formato de los campos con Zod (`loginSchema`); (4) se busca el usuario en MySQL via Prisma; (5) se compara la contraseña ingresada con el hash almacenado mediante `bcrypt.compare()`; (6) si es válida, NextAuth genera un token JWT firmado con `AUTH_SECRET`.

18.2 Autorización y Control de Acceso

La autorización opera en dos niveles:

- Nivel de ruta (`middleware.ts`): el middleware de Next.js intercepta TODAS las peticiones excepto rutas estáticas y de API. Verifica la existencia de una sesión JWT válida. Si no hay sesión, redirige a `/login`.
- Nivel de datos (`Route Handlers`): cada endpoint de API verifica la sesión mediante `auth()`. Adicionalmente, el endpoint de reportes filtra los datos según el rol y área del usuario autenticado.

18.3 Roles y Permisos

Funcionalidad	ADMIN	STAFF	TECNICO
Ver todos los reportes	✓	X (solo su área)	X (solo su área)
Crear reportes	✓	✓	✓
Cambiar estado de reporte	✓	✓	✓
Asignar área responsable	✓	✓	X
Asignar técnico	✓	✓	X
Eliminar reportes	✓	X	X
Ver analíticas	✓	✓	X
Gestionar personal (completo)	✓	X	X
Ver personal de área propia	✓	✓	X
Gestionar edificios/espacios	✓	X	X
Gestionar usuarios del sistema	✓	X	X

18.4 Seguridad de Contraseñas

Las contraseñas se hashean con `bcryptjs` usando 10 rounds de salt. `bcrypt` es un algoritmo de hashing de contraseñas diseñado específicamente para ser lento y resistente a ataques de fuerza bruta y rainbow tables. Con 10 rounds, el hashing tarda aproximadamente 100ms en hardware moderno, lo cual es negligible para el usuario pero significativamente costoso para un atacante que intente millones de intentos.

18.5 Fortalezas de Seguridad Identificadas

- Contraseñas hasheadas con bcryptjs (nunca se almacenan en texto plano).
- Tokens JWT firmados con secreto aleatorio (AUTH_SECRET).
- Validación de entrada en servidor con Zod antes de cualquier operación en base de datos.
- Filtrado de datos por área del usuario en la capa de API (no sólo en la UI).
- Variables de entorno para todos los secretos (.env excluido de Git).
- Restricción de eliminación de reportes exclusivamente al rol ADMIN.

18.6 Áreas de Mejora

- El AUTH_SECRET por defecto en docker-compose.yml ('andevia-dev-secret-change-in-production') debe obligatoriamente reemplazarse con un valor aleatorio de 32+ bytes antes de cualquier despliegue en producción.
- Implementar rate limiting en el endpoint de login para prevenir ataques de fuerza bruta.
- Agregar protección CSRF explícita (NextAuth v5 la incluye implícitamente para sus endpoints, pero los Route Handlers personalizados no).
- Considerar HTTPS obligatorio en producción mediante reverse proxy (Nginx o Traefik).

19. Pruebas y Calidad

19.1 Estado Actual

El proyecto en su estado actual no incluye una suite de pruebas automatizadas (tests unitarios, de integración o end-to-end). La validación funcional se realiza a través de pruebas manuales durante el desarrollo, apoyadas por las ventajas de tipado estático de TypeScript que previenen errores en tiempo de compilación.

19.2 Mecanismos de Calidad Existentes

- ESLint con configuración eslint-config-next para análisis estático de código.
- TypeScript strict mode para detección de errores en compilación.
- Zod para validación de esquemas tanto en cliente como en servidor.
- Prisma type-safe client que previene consultas SQL inválidas.

19.3 Estrategia de Testing Recomendada

Pruebas Unitarias

Se recomienda utilizar Vitest (compatible con el ecosistema Vite/Next.js) para pruebas unitarias de las funciones de utilería y lógica de negocio:

- Funciones en src/lib/utlis/ (formatDate, estadoLabels, exportReportesToExcel).
- Función normalizarNombre en src/lib/normalizar.ts.
- Función presetRange en src/lib/analiticas.ts.
- Schemas de validación Zod (casos válidos e inválidos).

Pruebas de Integración

Para los Route Handlers de la API, se recomienda usar MSW (Mock Service Worker) o supertest para probar los endpoints con una base de datos de prueba separada:

- POST /api/reportes: validar creación con datos correctos e incorrectos.
- PATCH /api/reportes/[id]: validar transiciones de estado y registro en LogAccion.
- GET /api/reportes: validar filtrado por área según rol.

Pruebas End-to-End

Playwright es la herramienta recomendada para pruebas E2E en proyectos Next.js:

- Flujo completo de login y acceso al dashboard.
- Creación de un reporte con evaluación y verificación de su aparición en el listado.
- Cambio de estado de reporte y verificación en bitácora.

20. Riesgos del Proyecto

Riesgo	Tipo	Impacto	Probabilidad	Estrategia de Mitigación
AUTH_SECRET vacío en producción	Seguridad	Alto	Media	Validar en entrypoint.sh que AUTH_SECRET no sea el valor por defecto. Documentar el proceso de generación con openssl.
Pérdida de datos por ausencia de backups	Infraestructura	Alto	Media	Implementar respaldo periódico del volumen Docker mysqldata. Usar mysqldump o herramientas de backup de MySQL.
Credenciales de DB expuestas en logs	Seguridad	Alto	Baja	Asegurar que DATABASE_URL no se imprima en logs de producción. Usar herramientas de gestión de secretos en producción.
Indisponibilidad de la DB durante el inicio de la app	Técnico	Alto	Baja	Ya mitigado: el entrypoint.sh implementa un loop de espera con netcat antes de ejecutar prisma db push.
Crecimiento del volumen de datos sin paginación suficiente	Rendimiento	Medio	Media	La API ya implementa paginación con límite de 50 registros. Monitorear y ajustar índices según crecimiento.
Ausencia de pruebas automatizadas	Técnico	Medio	Alta	Implementar suite de pruebas con Vitest y Playwright. Integrar en pipeline CI/CD.
Desactualización de NextAuth v5 beta	Técnico	Medio	Media	Monitorear el roadmap de Auth.js v5 y planificar migración a versión estable cuando se libere.
Resistencia del personal al cambio	Organizacional	Medio	Media	Capacitación al personal. Diseño de UI intuitivo. Soporte inicial durante la adopción.
Conflictos de datos en seed idempotente	Técnico	Bajo	Baja	La lógica de seed verifica existencia de usuarios antes de insertar. Para reset completo, eliminar el volumen Docker.
Uso de dependencias en versión beta	Técnico	Medio	Baja	NextAuth v5 beta es estable en sus funciones centrales. Mantener actualizaciones controladas.

21. Conclusiones

21.1 Beneficios Obtenidos

Andevia representa una solución integral que digitaliza y formaliza el proceso de gestión de incidencias en instalaciones físicas institucionales. Los beneficios principales son la trazabilidad completa del ciclo de vida de cada reporte, la asignación clara de responsabilidades mediante el sistema de áreas y técnicos, y la visibilidad en tiempo real del estado operativo de la infraestructura a través del dashboard y el módulo de analíticas.

21.2 Impacto Esperado

La implementación del sistema se espera que reduzca el tiempo promedio de resolución de incidencias al eliminar la fricción comunicacional entre quien reporta y quien atiende. La bitácora de auditoría automática proporciona evidencia objetiva para la evaluación del desempeño de las áreas de mantenimiento, facilitando la toma de decisiones basada en datos en lugar de percepciones subjetivas.

21.3 Aprendizajes Técnicos

El proyecto representa la aplicación práctica de tecnologías de vanguardia del ecosistema JavaScript/TypeScript: la arquitectura de React Server Components de Next.js 16, el patrón de validación isomórfica con Zod (mismos esquemas en cliente y servidor), la gestión de estado tipo-segura con Prisma, la contenerización reproducible con Docker y la autenticación moderna con NextAuth v5. Cada decisión tecnológica está justificada por las necesidades del dominio del problema.

21.4 Posibilidades de Crecimiento Futuro

- Módulo de notificaciones por correo electrónico (integración con Nodemailer o Resend) al crear, asignar y resolver reportes.
- Carga de fotografías de las incidencias en el formulario de reporte (Cloudinary ya está integrado como dependencia).
- Aplicación móvil progresiva (PWA) para uso durante inspecciones físicas sin conexión a internet.
- Módulo de mantenimiento preventivo con calendario de inspecciones programadas.
- Integración con directorio LDAP/Active Directory institucional para autenticación SSO.
- Exportación de analíticas en PDF para informes formales a autoridades.

22. Referencias

- Chiang, A. y equipo Auth.js. (2024). Auth.js v5 Documentation. <https://authjs.dev/>
- Grafar, S. y equipo Prisma. (2024). Prisma ORM Documentation. <https://www.prisma.io/docs>
- IFMA - International Facility Management Association. (2021). Facility Management Body of Knowledge. IFMA Press.
- Laudon, K. C., & Laudon, J. P. (2020). Sistemas de información gerencial (16.a ed.). Pearson Educación.
- MySQL AB. (2024). MySQL 8.0 Reference Manual. <https://dev.mysql.com/doc/refman/8.0/en/>
- Next.js Team - Vercel. (2024). Next.js 15 App Router Documentation. <https://nextjs.org/docs>
- OpenJS Foundation. (2024). Node.js 20 LTS Documentation. <https://nodejs.org/docs/latest-v20.x/api/>
- Oracle Corporation. (2024). MySQL 8.0 Reference Manual. <https://dev.mysql.com/doc/refman/8.0/en/>
- Pressman, R. S., & Maxim, B. R. (2020). Ingeniería del software: Un enfoque práctico (8.a ed.). McGraw-Hill.
- Tailwind Labs. (2024). Tailwind CSS v4 Documentation. <https://tailwindcss.com/docs>
- The Open Group. (2019). TOGAF Standard, Version 9.2: A Framework for Enterprise Architecture. The Open Group.
- Vercel Team. (2024). React Server Components Documentation. <https://react.dev/reference/rsc/server-components>
- shadcn. (2024). shadcn/ui Component Library. <https://ui.shadcn.com>
- Docker Inc. (2024). Docker Documentation. <https://docs.docker.com>
- Microsoft. (2024). TypeScript 5 Documentation. <https://www.typescriptlang.org/docs/>
- Colby F. (2024). Zod 4 Documentation. <https://zod.dev>

A. Matriz de Trazabilidad

Requerimiento	Caso de Uso	Historia de Usuario
RF-01 Autenticación	CU-01 Iniciar Sesión	HU-01, HU-02
RF-02 Control de acceso por roles	CU-01 Iniciar Sesión	HU-01
RF-03 Creación de reportes	CU-02 Crear Reporte	HU-03, HU-04
RF-04 Evaluaciones por categoría	CU-02 Crear Reporte	HU-05
RF-05 Guardado de borradores	CU-02 Crear Reporte	HU-04
RF-06 Listado y filtrado	CU-03 Consultar Reportes	HU-06, HU-07
RF-07 Visibilidad por área	CU-03 Consultar Reportes	HU-07
RF-08 Detalle de reporte	CU-03 Consultar Reportes	HU-08
RF-09 Cambio de estado	CU-04 Actualizar Estado	HU-10, HU-11
RF-10 Asignación de área	CU-03 Asignar Área y Técnico	HU-08
RF-11 Asignación de técnico	CU-03 Asignar Área y Técnico	HU-09
RF-12 Bitácora de auditoría	CU-04 Actualizar Estado	HU-12
RF-13 Gestión de espacios	CU-08 Gestionar Espacios	HU-18
RF-14 Gestión de personal	CU-07 Gestionar Personal	HU-20
RF-15 Registro automático solicitante	CU-02 Crear Reporte	HU-03
RF-16 Analíticas y KPIs	CU-04 Consultar Analíticas	HU-13, HU-14, HU-15, HU-16
RF-17 Rango de fechas en analíticas	CU-04 Consultar Analíticas	HU-14
RF-18 Exportación a Excel	CU-10 Exportar a Excel	HU-17
RF-19 Dashboard con estadísticas	CU-03 Consultar Reportes	HU-19
RF-20 Gestión de usuarios	CU-09 Gestionar Usuarios	HU-19